

Database Technologies – Assignment 11

1) Create a view that shows all of the customers who have the highest ratings.

```
D6_96342_Jagdish@localhost> CREATE VIEW high_rating_cust AS SELECT * FROM customers WHERE rating = (SELECT MAX(rating) FROM customers);
Query OK, 0 rows affected (0.003 sec)

D6_96342_Jagdish@localhost> SELECT * FROM high_rating_cust;
+-----+-----+-----+-----+-----+
| cnum | cname | city | rating | snum |
+-----+-----+-----+-----+-----+
| 2004 | Grass | Berlin | 300 | 1002 |
| 2008 | Cisneros | San Jose | 300 | 1007 |
+-----+-----+-----+-----+-----+
2 rows in set (0.002 sec)
```

2) Create a view that shows the number of salespeople in each city.

```
D6_96342_Jagdish@localhost> CREATE view city_sp_count AS SELECT city, COUNT(snum) count FROM salespeople GROUP BY city;
Query OK, 0 rows affected (0.003 sec)

D6_96342_Jagdish@localhost> SELECT * FROM city_sp_count;
+-----+-----+
| city | count |
+-----+-----+
| London | 2 |
| San Jose | 1 |
| Barcelona | 1 |
| New York | 1 |
+-----+-----+
4 rows in set (0.004 sec)
```

3) Create a view that shows the average and total orders for each salesperson after his or her name. Assume all names are unique.

```
D6_96342_Jagdish@localhost> CREATE VIEW sp_order_summary AS SELECT s.sname, AVG(o.amt) avg, SUM(o.amt) sum FROM salespeople s JOIN orders o ON s.snum = o.snum GROUP BY sname;
Query OK, 0 rows affected (0.003 sec)

D6_96342_Jagdish@localhost> SELECT * FROM sp_order_summary;
+-----+-----+-----+
| sname | avg | sum |
+-----+-----+-----+
| Rifkin | 558.425000 | 1116.85 |
| Peel | 5127.356667 | 15382.07 |
| Motika | 1900.100000 | 1900.10 |
| Serres | 1848.716667 | 5546.15 |
| Axelrod | 1713.230000 | 1713.23 |
+-----+-----+-----+
5 rows in set (0.001 sec)
```

4) Create a view that shows each salesperson with multiple customers.

```
D6_96342_Jagdish@localhost> CREATE VIEW sp_customers AS SELECT s.sname, COUNT(c.cnum) total_customers FROM salespeople
s JOIN customers c ON s.snum = c.snum GROUP BY s.sname;
Query OK, 0 rows affected (0.004 sec)

D6_96342_Jagdish@localhost> SELECT * FROM sp_customers;
```

sname	total_customers
Peel	2
Axelrod	1
Serres	2
Rifkin	1
Motika	1

5 rows in set (0.002 sec)

Changing Values through Views.

5) Which of these views are updateable (will allow DML operations)?

#1 Create View Dailyorders as Select
Distinct cnum, snum, onum, odate from Orders;

Ans -> Yea, this is a simple view with same name of columns as original table. Hence, it is updateable (will allow DML operations).

#2 Create View Custotals as Select cname, Sum
(amt) Sum_Amt from Orders, Customers
where Orders.cnum=Customers.cnum
Group by cname;

Ans -> No, this is complex view having computed column Sum (amt). Hence, it is not updateable (will not allow DML operations).

#3 Create view Thirdorders as Select *
from Dailyorders where
odate='1990-10-03';

Ans -> Yes, this is a simple view with all the
columns of original table. Hence, it is updateable (will
allow DML operations).

#4 Create view Nullcities as Select snum, sname, city
from Salespeople where city is NULL
OR sname BETWEEN 'A' and 'MZ';

Ans -> Yes, this is a simple view which has no
computed columns. Hence, it is updateable (will allow
DML operations).

6) Create a view of the Salespeople table called
Commissions. This view will include only
the snum and comm fields. Through this view,
someone could enter or change commissions, but
only to values between .10 and .20.

```
[D6_96342_Jagdish@localhost> CREATE VIEW commission AS SELECT snum, comm FROM salespeople WHERE comm > .10 AND comm < .2]
0 WITH CHECK OPTION;
Query OK, 0 rows affected (0.003 sec)

[D6_96342_Jagdish@localhost> SELECT * FROM commission;

+-----+-----+
| snum | comm |
+-----+-----+
| 1001 | 0.12 |
| 1002 | 0.13 |
| 1004 | 0.11 |
| 1007 | 0.15 |
+-----+-----+
4 rows in set (0.002 sec)
```